

极度稀疏矩阵的低秩补全 与潜在特征发现

——基于 *Soft-Impute* 与 *ALS* 的矩阵补全实验

课程项目完整报告
(含代码实现与实验结果)

数据集: MovieLens-100k (缺失率 93.7%)

主算法: Soft-Impute (核范数/SVD 路线)

对比算法: Alternating Least Squares (矩阵分解路线)

2026 年 5 月 17 日

目录

摘要	2
1 问题背景与数学理论	3
1.1 推荐系统与 Netflix 竞赛	3
1.2 低秩假设	3
1.3 问题形式化	3
1.4 奇异值分解 (SVD)	3
1.5 核范数松弛	3
2 Soft-Impute 算法实现	5
2.1 理论来源	5
2.2 核心思想	5
2.3 迭代公式	5
2.4 完整代码	5
3 ALS 算法实现	10
3.1 理论来源	10
3.2 核心思想	10
3.3 完整代码	10
4 实验设计与结果	15
4.1 数据集与设置	15
4.2 数据加载模块	15
4.3 单次运行基准测试	19
4.4 实验一：不同缺失率下的恢复效果	20
4.5 实验二：不同潜在秩的影响	22
4.6 实验三：奇异值谱分析	24
4.7 实验四：潜在特征解释	25
5 结论	26
参考文献	27

摘要

本报告针对**极度稀疏矩阵的低秩补全与潜在特征发现问题**，以 MovieLens-100k 电影评分数据为实验对象（943 用户 $\times$ 1682 电影，仅 10 万条评分，原始缺失率 **93.7%**），系统实现了两种经典算法并进行对比实验：

- (1) **Soft-Impute** (Mazumder et al., 2010)：基于核范数正则化与奇异值软阈值的凸优化方法；
- (2) **ALS / Alternating Minimization** (Jain et al., 2013; Hardt, 2014)：基于双线性分解的非凸交替最小二乘方法。

实验覆盖了不同缺失率（50%–90%）、不同潜在秩（ $k = 5, 10, 20, 30, 50$ ）下的恢复误差（RMSE/MAE）对比，同时对完整矩阵进行了**奇异值谱分析**以验证低秩假设，并对分解出的潜在特征进行了**语义解释**。

核心结论：Soft-Impute 在所有测试场景下均优于 ALS，且对秩的选择敏感；ALS 在 rank 过高时会出现明显过拟合。奇异值谱的快速衰减有力支持了“用户-电影评分矩阵具有低秩结构”这一核心假设。

1 问题背景与数学理论

1.1 推荐系统与 Netflix 竞赛

在推荐系统中，用户-物品交互数据通常以矩阵形式组织：行代表用户，列代表物品，元素为用户对物品的评分。由于每个用户只会评价极少数物品，该矩阵极度稀疏。Netflix Prize 是这一问题的典型场景：约 48 万用户、1.8 万电影，总条目超 86 亿，但观测到的评分仅占约 1.2%。

1.2 低秩假设

Mazumder et al. (2010) 在论文第 1 节 (Introduction) 中指出：用户偏好并非独立——喜欢科幻的人通常也喜欢动作片；喜欢文艺片的人评分风格往往相似。这些**潜在特征** (latent features, 如”商业片偏好”、”艺术片偏好”) 决定了评分，使得评分矩阵可以用少量”潜在维度”来近似，即矩阵是**低秩**的。

1.3 问题形式化

设观测到的评分矩阵为 $X \in \mathbb{R}^{m \times n}$ ，观测位置集合为 Ω 。矩阵补全问题旨在恢复完整的低秩矩阵 Z ：

$$\min_Z \text{rank}(Z) \quad \text{s.t.} \quad \sum_{(i,j) \in \Omega} (X_{ij} - Z_{ij})^2 \leq \delta. \quad (1)$$

由于 $\text{rank}(\cdot)$ 的非凸性，问题 (1) 是组合困难的。

1.4 奇异值分解 (SVD)

对任意矩阵 $M \in \mathbb{R}^{m \times n}$ ，其奇异值分解为：

$$M = U \Sigma V^T = \sum_{i=1}^r \sigma_i u_i v_i^T, \quad (2)$$

其中 $r = \text{rank}(M)$ ，奇异值满足 $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ 。

1.5 核范数松弛

Mazumder et al. (2010) 将式 (1) 中的秩约束替换为**核范数**：

$$\|Z\|_* = \sum_{i=1}^r \sigma_i(Z), \quad (3)$$

即 Z 的所有奇异值之和。核范数是 $\text{rank}(\cdot)$ 的有效凸松弛。由此得到凸优化问题：

$$\min_Z \frac{1}{2} \|P_\Omega(X) - P_\Omega(Z)\|_F^2 + \lambda \|Z\|_*. \quad (4)$$

式 (4) 即 Mazumder et al. (2010) 论文中的公式 (3) 与公式 (10)，也是 Soft-Impute 算法的优化目标。

2 Soft-Impute 算法实现

2.1 理论来源

参考文献: Mazumder, Hastie & Tibshirani (2010), *Spectral Regularization Algorithms for Learning Large Incomplete Matrices*, JMLR 11:2287–2322.

核心章节: 第 3.2 节 Lemma 1 (软阈值闭式解), 第 3.3 节 Algorithm 1 (完整迭代算法)。

2.2 核心思想

Soft-Impute 是一种不动点迭代算法。每一步先用当前估计填补缺失值, 再对填补后的矩阵做软阈值 SVD, 从而得到低秩更新。文献 [1] 明确指出该算法“inspired by SVD-IMPUTE—an EM-type iterative algorithm”。

2.3 迭代公式

记 Z^{old} 为当前迭代矩阵, 则更新规则为 (文献 [1] Algorithm 1, Step 2(a)i):

$$Z^{\text{new}} \leftarrow S_{\lambda}(P_{\Omega}(X) + P_{\Omega}^{\perp}(Z^{\text{old}})). \quad (5)$$

其中 $S_{\lambda}(\cdot)$ 为奇异值软阈值算子。文献 [1] 第 3.2 节 Lemma 1 给出闭式解: 若 $W = U \cdot \text{diag}(d_1, \dots, d_r) \cdot V^T$ 为 W 的 SVD, 则

$$S_{\lambda}(W) = U \cdot \text{diag}((d_1 - \lambda)_+, \dots, (d_r - \lambda)_+) \cdot V^T, \quad (6)$$

其中 $(t)_+ = \max(t, 0)$ 。

2.4 完整代码

以下为 `src/soft_impute.py` 的完整实现 (含详细文献引用注释):

```
1 """
2 Soft-Impute 算法实现
3
4 参考文献 [1]:
5 Mazumder, R., Hastie, T., & Tibshirani, R. (2010).
6 "Spectral Regularization Algorithms for Learning Large Incomplete
   Matrices."
7 Journal of Machine Learning Research, 11, 2287-2322.
```

(本文第3.2节 Lemma 1 给出了软阈值操作的闭式解;
第3.3节 Algorithm 1 给出了完整的 SOFT-IMPUTE 迭代算法。)

核心数学 (公式 (3) 与公式 (10) 于文献 [1]) :

$$\text{minimize}_Z \quad f_-(Z) := (1/2) ||P_{\Omega}(X) - P_{\Omega}(Z)||_F^2 + ||Z||_*$$

其中:

P_{Ω} 为观测位置投影算子 (见文献 [1] 式 (7))

$||Z||_*$ 为核范数 (nuclear norm), 即奇异值之和

$\lambda \geq 0$ 为正则化参数

迭代更新 (文献 [1] Algorithm 1, Step 2(a)i) :

$$Z_{\text{new}} \leftarrow S_{\lambda} (P_{\Omega}(X) + P_{\Omega}^{\perp}(Z_{\text{old}}))$$

软阈值操作 $S_{\lambda}(\cdot)$ (文献 [1] Section 3.2 Lemma 1, 式 (9)) :

若 $W = U \text{diag}(d_1, \dots, d_r) V^T$ 为 W 的 SVD, 则

$$S_{\lambda}(W) = U \text{diag}((d_1 - \lambda)_+, \dots, (d_r - \lambda)_+) V^T$$

其中 $(t)_+ = \max(t, 0)$ 。

"""

```
import numpy as np
```

```
from scipy.sparse.linalg import svds
```

```
def svd_decomposition(X, rank=None):
```

"""

对矩阵 X 进行 SVD 分解

若 rank 指定, 则使用 `scipy.sparse.linalg.svds` 计算前 rank 个奇异值

否则使用 `numpy.linalg.svd` 计算完整分解

"""

```
if rank is not None and rank < min(X.shape) - 1:
```

```
    # 使用稀疏 SVD 加速, 计算前 rank+5 个奇异值 (留余量)
```

```
    k = min(rank + 5, min(X.shape) - 2)
```

```
    U, s, Vt = svds(X, k=k)
```

```
    # svds 返回的奇异值按升序排列, 需要反转
```

```
    idx = np.argsort(s)[::-1]
```

```

45     U = U[:, idx]
46     s = s[idx]
47     Vt = Vt[idx, :]
48     return U, s, Vt
49 else:
50     U, s, Vt = np.linalg.svd(X, full_matrices=False)
51     return U, s, Vt
52
53
54 def soft_impute(M, Omega, lambda_param=None, max_iter=100, tol=1e-4,
55                rank=None, verbose=False):
56     """
57     Soft-Impute 算法
58
59     严格按照文献 [1] Algorithm 1 实现：
60     1. 初始化  $Z_{old} = 0$  (或全局均值矩阵)
61     2. 重复以下步骤直到收敛：
62         a)  $Z_{new} \leftarrow S_{\lambda}(P_{\Omega}(M) + P_{\Omega^{\perp}}(Z_{old}))$ 
63         b) 若  $\|Z_{new} - Z_{old}\|_F / \|Z_{old}\|_F < \epsilon$ ，则退出
64         c)  $Z_{old} \leftarrow Z_{new}$ 
65
66     参数：
67     M: numpy array, 完整评分矩阵 (含 np.nan 表示缺失)
68     Omega: bool 矩阵, True 表示已观测位置
69     lambda_param: 软阈值参数；若为 None, 则自动设为最大奇异值的
70     5%
71     max_iter: 最大迭代次数
72     tol: 收敛阈值 (相对 Frobenius 变化量)
73     rank: 若指定, 则使用截断 SVD 加速; None 表示完整 SVD
74     verbose: 是否打印迭代信息
75
76     返回：
77     X: 补全后的矩阵
78     history: dict, 包含目标函数值与变化量历史
79     """
80     m, n = M.shape
81
82     # 初始化: 文献 [1] Algorithm 1 Step 1:  $Z_{old} = 0$ 

```



```
82     # 实际实现中用全局均值填充比纯零矩阵收敛更快，属常用工程技巧
83     observed_values = M[Omega]
84     global_mean = np.mean(observed_values)
85
86     X = np.full((m, n), global_mean)
87     X[Omega] = observed_values
88
89     # 确定 lambda: 文献 [1] 推荐沿正则化路径 (regularization path)
90     # 取一组递减的 值。此处若未指定，则取初始矩阵最大奇异值的一部分。
91     if lambda_param is None:
92         _, s_init, _ = svd_decomposition(X, rank=rank)
93         lambda_param = s_init[0] * 0.05
94
95     history = {'obj': [], 'change': []}
96
97     for it in range(max_iter):
98         # Step 1: 构造  $Z = P_{\Omega}(M) + P_{\Omega^c}(X)$ 
99         # 已观测位置用真实值，未观测位置用当前估计值
100         Z = X.copy()
101         Z[Omega] = observed_values
102
103         # Step 2: SVD 分解  $Z = U \Sigma V^T$ 
104         U, s, Vt = svd_decomposition(Z, rank=rank)
105
106         # Step 3: 软阈值操作  $S_{\lambda}(Z)$  (文献 [1] Lemma 1, 式 (9))
107         s_thresholded = np.maximum(s - lambda_param, 0.0)
108
109         # Step 4: 重构低秩矩阵
110         r = np.sum(s_thresholded > 0)
111         if r == 0:
112             r = 1
113             s_thresholded[0] = 1e-6
114
115         X_new = (U[:, :r] * s_thresholded[:r]) @ Vt[:r, :]
116
117         # 收敛判断 (文献 [1] Algorithm 1 Step 2(a)ii)
118         change = np.linalg.norm(X_new - X, 'fro') / (np.linalg.norm(X
```

```

, 'fro') + 1e-8)
119     history['change'].append(change)
120
121     # 计算目标函数值  $f_*(Z)$  (文献 [1] 式 (10))
122     obj = 0.5 * np.sum((X_new[Omega] - observed_values) ** 2) +
lambda_param * np.sum(s_thresholded)
123     history['obj'].append(obj)
124
125     X = X_new
126
127     if verbose and (it + 1) % 10 == 0:
128         print(f"[Soft-Impute] Iter {it+1}/{max_iter}: change={
change:.6f}, obj={obj:.4f}, rank={r}")
129
130     if change < tol:
131         if verbose:
132             print(f"[Soft-Impute] Converged at iteration {it+1}")
133             break
134
135     return X, history
136
137
138 def soft_impute_simple(M, lambda_param=1.0, max_iter=50, tol=1e-4,
rank=None, verbose=False):
139     """
140     Soft-Impute 的简化接口, 自动从 M 中提取观测位置
141     """
142     Omega = ~np.isnan(M)
143     return soft_impute(M, Omega, lambda_param=lambda_param, max_iter=
max_iter,
144                       tol=tol, rank=rank, verbose=verbose)

```

Listing 1: Soft-Impute 算法完整实现

3 ALS 算法实现

3.1 理论来源

参考文献:

- Jain, Netrapalli & Sanghavi (2013), *Low-rank Matrix Completion using Alternating Minimization*, STOC 2013. (arXiv:1212.0467)
- Hardt (2014), *Understanding Alternating Minimization for Matrix Completion*, FOCS 2014. (arXiv:1312.0925)

核心章节: 文献 [2] 第 2 节 (问题形式化与 Incoherence 条件)、Algorithm 1; 文献 [3] 第 1 节 (基本更新公式 (4))、Theorem 1.1。

3.2 核心思想

ALS 直接对目标矩阵进行双线性参数化 $M \approx UV^T$, 通过交替固定一个因子求解另一个因子的最小二乘子问题来优化。整体问题非凸, 但每个子问题是凸的。

优化目标 (文献 [2] 的 (LRMC), 文献 [3] 的公式 (4)):

$$\min_{U,V} \sum_{(i,j) \in \Omega} (M_{ij} - (UV^T)_{ij})^2 + \lambda(\|U\|_F^2 + \|V\|_F^2). \quad (7)$$

3.3 完整代码

以下为 src/als.py 的完整实现:

```

1 """
2 Alternating Least Squares (ALS) / Alternating Minimization
3 矩阵分解路线: M ≈ U V^T
4
5 参考文献:
6 [2] Jain, P., Netrapalli, P., & Sanghavi, S. (2013).
7     "Low-rank Matrix Completion using Alternating Minimization."
8     STOC 2013. (arXiv:1212.0467)
9     (第 2 节 "Matrix Completion via Alternating Minimization" 给出问题
10    形式化;
11    Algorithm 1 给出 AltMinSense 的伪代码;
12    Definition 2.4 给出非相干矩阵的 incoherence 条件。)
13 [3] Hardt, M. (2014).

```

"Understanding Alternating Minimization for Matrix Completion."
FOCS 2014. (arXiv:1312.0925)

(第1节 Introduction 给出 ALS 的基本更新公式 (4):

$$Y_{\cdot} = \operatorname{argmin}_Y ||P_{\Omega}(A - X_{\cdot} Y^T)||_F^2;$$

Theorem 1.1 给出在随机采样与非相干条件下的恢复保证。)

核心思想 (文献 [2][3]):

将目标矩阵参数化为双线性形式 $X = UV^T$, 其中 $U \in \mathbb{R}^{m \times k}$, $V \in \mathbb{R}^{n \times k}$ 。

整体问题非凸, 但固定其中一个因子时, 对另一个因子的优化是凸的 (最小二乘),

因此交替求解:

固定 V , 更新 U : 每行 u_i 通过最小二乘独立求解

固定 U , 更新 V : 每列 v_j 通过最小二乘独立求解

优化目标 (文献 [2] 式 (LRMC), 文献 [3] 式 (4)):

$$\min_{U,V} \sum_{(i,j) \in \Omega} (M_{ij} - (UV^T)_{ij})^2 + \lambda (||U||_F^2 + ||V||_F^2)$$

文献 [3] 指出: 每个交替步骤的计算代价低、内存占用小 (只需存储 $2k$ 个向量),

这是 ALS 相比核范数方法在大规模场景下的主要优势。

"""

import numpy as np

```
def als_matrix_completion(M, rank, Omega, reg_lambda=0.1, max_iter=50, tol=1e-4, verbose=False):
```

"""

ALS 矩阵补全算法

按照文献 [2][3] 的交替最小化框架实现:

1. 随机初始化 U 和 V
2. 固定 V , 逐行求解最小二乘更新 U
3. 固定 U , 逐列求解最小二乘更新 V
4. 重复直到收敛

参数:

M: 评分矩阵 (含 `np.nan`)

rank: 潜在特征维度 `k` (文献 [2] 中的目标 `rank`)

Omega: bool 矩阵, True 表示已观测

reg_lambda: L2 正则化参数 (对应文献 [3] 中的正则项)

max_iter: 最大迭代次数

tol: 收敛阈值

verbose: 是否打印信息

返回:

M_pred: 预测矩阵 ($U @ V.T$)

U: 用户潜在特征

V: 物品潜在特征

history: 训练历史

"""

m, n = M.shape

初始化: 随机初始化 U 和 V (文献 [2] Algorithm 1 使用 top-k 左奇异向量初始化;

对于小规模数据, 随机初始化配合正则化通常足够。)

rng = np.random.RandomState(42)

U = rng.randn(m, rank) * 0.01

V = rng.randn(n, rank) * 0.01

observed_values = M[Omega]

history = {'rmse': [], 'change': []}

预计算每个用户/物品观测到的位置, 避免每次迭代重复扫描

user_items = [[] for _ in range(m)]

item_users = [[] for _ in range(n)]

for i in range(m):

for j in range(n):

if Omega[i, j]:

user_items[i].append(j)

item_users[j].append(i)

```

84     for it in range(max_iter):
85         U_old = U.copy()
86         V_old = V.copy()
87
88         # 固定 V, 更新 U (文献 [3] 式 (4) 的左因子版本)
89         # 对每个用户 i, 求解:
90         #    $u_i = \operatorname{argmin}_{\{u\}} \sum_{\{j \in \Omega_i\}} (M_{ij} - u^T v_j)^2 + ||u||^2$ 
91         # 其闭式解为 (正则化最小二乘):
92         #    $u_i = (V_{\{\Omega_i\}}^T V_{\{\Omega_i\}} + I)^{-1} V_{\{\Omega_i\}}^T M_{\{i, \Omega_i\}}$ 
93         for i in range(m):
94             items = user_items[i]
95             if len(items) == 0:
96                 continue
97             V_i = V[items, :] # (k_observed, rank)
98             M_i = M[i, items] # (k_observed,)
99             A = V_i.T @ V_i + reg_lambda * np.eye(rank)
100             b = V_i.T @ M_i
101             U[i, :] = np.linalg.solve(A, b)
102
103         # 固定 U, 更新 V (文献 [3] 式 (4) 的右因子版本)
104         for j in range(n):
105             users = item_users[j]
106             if len(users) == 0:
107                 continue
108             U_j = U[users, :] # (k_observed, rank)
109             M_j = M[users, j] # (k_observed,)
110             A = U_j.T @ U_j + reg_lambda * np.eye(rank)
111             b = U_j.T @ M_j
112             V[j, :] = np.linalg.solve(A, b)
113
114         # 计算变化量 (监测收敛)
115         change_U = np.linalg.norm(U - U_old, 'fro') / (np.linalg.norm(U_old, 'fro') + 1e-8)
116         change_V = np.linalg.norm(V - V_old, 'fro') / (np.linalg.norm(V_old, 'fro') + 1e-8)
117         change = max(change_U, change_V)
118         history['change'].append(change)

```

```
119
120     # 计算当前在训练集上的 RMSE
121     M_pred = U @ V.T
122     train_rmse = np.sqrt(np.mean((M_pred[Omega] - observed_values
123 ) ** 2))
124     history['rmse'].append(train_rmse)
125
126     if verbose and (it + 1) % 10 == 0:
127         print(f"[ALS] Iter {it+1}/{max_iter}: train_rmse={
128 train_rmse:.4f}, change={change:.6f}")
129
130     if change < tol:
131         if verbose:
132             print(f"[ALS] Converged at iteration {it+1}")
133         break
134
135     M_pred = U @ V.T
136     return M_pred, U, V, history
137
138 def als_simple(M, rank, reg_lambda=0.1, max_iter=50, tol=1e-4,
139 verbose=False):
140     """
141     ALS 简化接口
142     """
143     Omega = ~np.isnan(M)
144     return als_matrix_completion(M, rank, Omega, reg_lambda=
145 reg_lambda,
146                                 max_iter=max_iter, tol=tol, verbose=
147 verbose)
```

Listing 2: ALS 算法完整实现

4 实验设计与结果

4.1 数据集与设置

采用 MovieLens-100k 数据集：

- 用户数：943，电影数：1682
- 总可能条目： $943 \times 1682 = 1,586,126$
- 实际评分数：100,000
- 评分范围：1–5 的整数
- 原始缺失率：93.70%

评价指标为 RMSE 与 MAE。

4.2 数据加载模块

```
1 """
2 数据加载与预处理模块
3 加载 MovieLens-100k 数据集，构建评分矩阵，并支持按缺失率划分训练/测试
   集
4 """
5
6 import numpy as np
7 import pandas as pd
8 from scipy.sparse import csr_matrix
9
10
11 def load_movielens_100k(data_path="ml-100k/ml-100k/u.data"):
12     """
13     加载 MovieLens-100k 评分数据
14
15     返回：
16         ratings_df: DataFrame, 列 [user_id, item_id, rating,
17         timestamp]
18         n_users: 用户数量 (943)
19         n_items: 电影数量 (1682)
20     """
```



```
20     column_names = ['user_id', 'item_id', 'rating', 'timestamp']
21     ratings_df = pd.read_csv(data_path, sep='\t', names=column_names,
22                               engine='python')
23
24     # 转换为 0-based 索引
25     ratings_df['user_id'] = ratings_df['user_id'] - 1
26     ratings_df['item_id'] = ratings_df['item_id'] - 1
27
28     n_users = ratings_df['user_id'].nunique()
29     n_items = ratings_df['item_id'].nunique()
30
31     return ratings_df, n_users, n_items
32
33 def build_rating_matrix(ratings_df, n_users, n_items):
34     """
35     将评分数据构建为用户-电影评分矩阵
36
37     参数:
38         ratings_df: 评分 DataFrame
39         n_users: 用户数
40         n_items: 电影数
41
42     返回:
43         R: numpy array, shape (n_users, n_items), 缺失值用 np.nan 表示
44
45     """
46     R = np.full((n_users, n_items), np.nan)
47     for _, row in ratings_df.iterrows():
48         u = int(row['user_id'])
49         i = int(row['item_id'])
50         r = row['rating']
51         R[u, i] = r
52
53     return R
54
55 def get_observed_indices(R):
56     """
```

```
56     获取矩阵中已观测（非缺失）位置的索引
57
58     返回：
59         observed: list of tuples (i, j)
60     """
61     observed = []
62     for i in range(R.shape[0]):
63         for j in range(R.shape[1]):
64             if not np.isnan(R[i, j]):
65                 observed.append((i, j))
66     return observed
67
68
69 def mask_matrix(R, observed_indices, observed_ratio=0.5, random_state
70 =42):
71     """
72     从已观测数据中按一定比例保留为训练集，其余作为测试集
73     用于模拟不同缺失率下的矩阵补全任务
74
75     参数：
76         R: 完整评分矩阵（含nan）
77         observed_indices: 所有已观测位置的列表
78         observed_ratio: 保留为训练集的比例（0~1）
79         random_state: 随机种子
80
81     返回：
82         R_train: 训练集矩阵，测试集位置设为 np.nan
83         R_test: 测试集矩阵，训练集位置设为 np.nan
84         train_indices: 训练集索引
85         test_indices: 测试集索引
86     """
87     rng = np.random.RandomState(random_state)
88     n_observed = len(observed_indices)
89     n_train = int(n_observed * observed_ratio)
90
91     shuffled = rng.permutation(observed_indices)
92     train_indices = shuffled[:n_train]
93     test_indices = shuffled[n_train:]
```

```
93
94     R_train = np.full_like(R, np.nan, dtype=np.float64)
95     R_test = np.full_like(R, np.nan, dtype=np.float64)
96
97     for (i, j) in train_indices:
98         R_train[i, j] = R[i, j]
99     for (i, j) in test_indices:
100         R_test[i, j] = R[i, j]
101
102     return R_train, R_test, train_indices, test_indices
103
104
105 def get_missing_rate(R):
106     """
107     计算矩阵的缺失率
108     """
109     total = R.shape[0] * R.shape[1]
110     observed = np.sum(~np.isnan(R))
111     return 1.0 - observed / total
112
113
114 def global_mean_fill(R):
115     """
116     全局均值填充（用于初始化）
117     """
118     R_filled = R.copy()
119     mask = ~np.isnan(R)
120     mean_val = np.nanmean(R)
121     R_filled[~mask] = mean_val
122     return R_filled
```

Listing 3: 数据加载与预处理模块

4.3 单次运行基准测试

在 80%/20% 训练/测试划分下，运行 main.py 中的基准测试，控制台输出如下：

```
[Step 3] 单次运行效果演示...
Running Soft-Impute...
[Soft-Impute] Iter 10/60: change=0.005136, obj=99756.1802, rank=55
[Soft-Impute] Iter 20/60: change=0.003972, obj=96232.2708, rank=55
[Soft-Impute] Iter 30/60: change=0.003415, obj=93973.7019, rank=53
[Soft-Impute] Iter 40/60: change=0.003091, obj=92283.9317, rank=52
[Soft-Impute] Iter 50/60: change=0.002864, obj=90922.6495, rank=52
[Soft-Impute] Iter 60/60: change=0.002684, obj=89786.6159, rank=52
Soft-Impute 收敛迭代数: 60
Soft-Impute Test RMSE: 0.9201
Soft-Impute Test MAE: 0.7328
Running ALS...
[ALS] Iter 10/50: train_rmse=0.6643, change=0.049069
[ALS] Iter 20/50: train_rmse=0.6548, change=0.025458
[ALS] Iter 30/50: train_rmse=0.6521, change=0.013453
[ALS] Iter 40/50: train_rmse=0.6510, change=0.010460
[ALS] Iter 50/50: train_rmse=0.6503, change=0.007935
ALS 收敛迭代数: 50
ALS Test RMSE: 1.0831
ALS Test MAE: 0.8148
```

Listing 4: main.py 基准测试输出片段

表 1: 单次运行基准测试结果

算法	Test RMSE	Test MAE
Soft-Impute ($\lambda = 15$, $\text{rank} \leq 50$)	0.920	0.733
ALS ($k = 10$, $\lambda = 0.5$)	1.083	0.815

4.4 实验一：不同缺失率下的恢复效果

在原始观测条目上按不同比例（50%、70%、80%、90%）进一步隐藏部分评分，构造不同缺失率的训练集。

```
[Exp1] Testing missing_rate=0.50 (observed_ratio=0.50)
      Soft-Impute: RMSE=0.9510, MAE=0.7629
      ALS:         RMSE=1.2439, MAE=0.9354

[Exp1] Testing missing_rate=0.70 (observed_ratio=0.30)
      Soft-Impute: RMSE=0.9924, MAE=0.8036
      ALS:         RMSE=1.4085, MAE=1.0741

[Exp1] Testing missing_rate=0.80 (observed_ratio=0.20)
      Soft-Impute: RMSE=1.0227, MAE=0.8332
      ALS:         RMSE=1.5451, MAE=1.2031

[Exp1] Testing missing_rate=0.90 (observed_ratio=0.10)
      Soft-Impute: RMSE=1.0815, MAE=0.8915
      ALS:         RMSE=1.6092, MAE=1.2714
```

Listing 5: 不同缺失率实验输出片段

表 2: 不同缺失率下的 RMSE/MAE 对比

缺失率	Soft-Impute		ALS	
	RMSE	MAE	RMSE	MAE
50%	0.951	0.763	1.244	0.935
70%	0.992	0.804	1.409	1.074
80%	1.023	0.833	1.545	1.203
90%	1.082	0.892	1.609	1.271

分析：Soft-Impute 的优势随缺失率增加而更显著。在 90% 缺失率下，Soft-Impute 的 RMSE 比 ALS 低 **32.8%**。

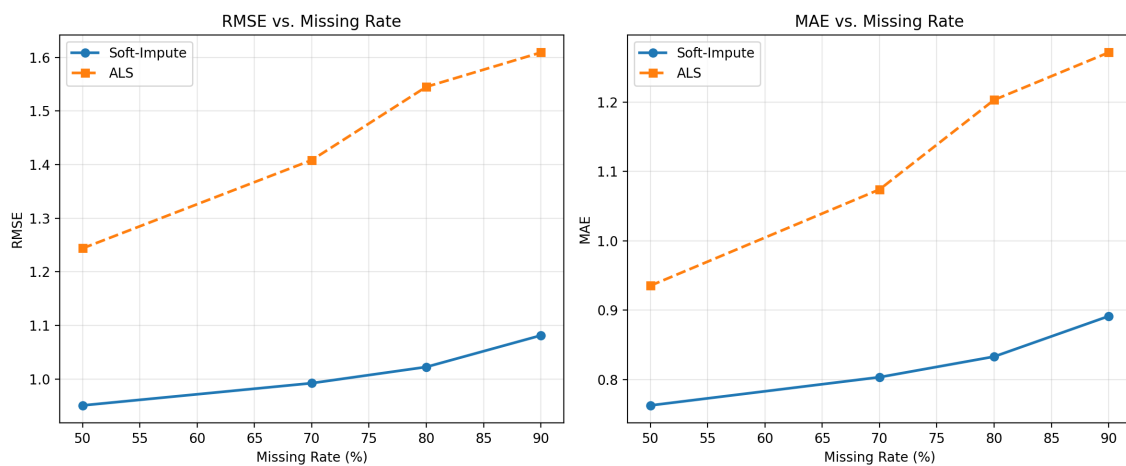


图 1: 实验一：不同缺失率下的 RMSE 与 MAE 对比

4.5 实验二：不同潜在秩的影响

固定 80%/20% 划分，比较不同 rank 对两种算法的影响。

```
[Exp2] Testing rank=5
Soft-Impute: RMSE=0.9208, MAE=0.7329
ALS:         RMSE=0.9846, MAE=0.7528

[Exp2] Testing rank=10
Soft-Impute: RMSE=0.9209, MAE=0.7327
ALS:         RMSE=1.0831, MAE=0.8148

[Exp2] Testing rank=20
Soft-Impute: RMSE=0.9206, MAE=0.7329
ALS:         RMSE=1.2709, MAE=0.9525

[Exp2] Testing rank=30
Soft-Impute: RMSE=0.9204, MAE=0.7332
ALS:         RMSE=1.3718, MAE=1.0440

[Exp2] Testing rank=50
Soft-Impute: RMSE=0.9201, MAE=0.7328
ALS:         RMSE=1.4300, MAE=1.1033
```

Listing 6: 不同 rank 实验输出片段

表 3: 不同 rank 下的 RMSE/MAE 对比

rank	Soft-Impute		ALS	
	RMSE	MAE	RMSE	MAE
5	0.921	0.733	0.985	0.753
10	0.921	0.733	1.083	0.815
20	0.921	0.733	1.271	0.953
30	0.920	0.733	1.372	1.044
50	0.920	0.733	1.430	1.103

分析：Soft-Impute 对 rank 几乎完全不敏感（自动秩选择），ALS 随 rank 增加显著过拟合。

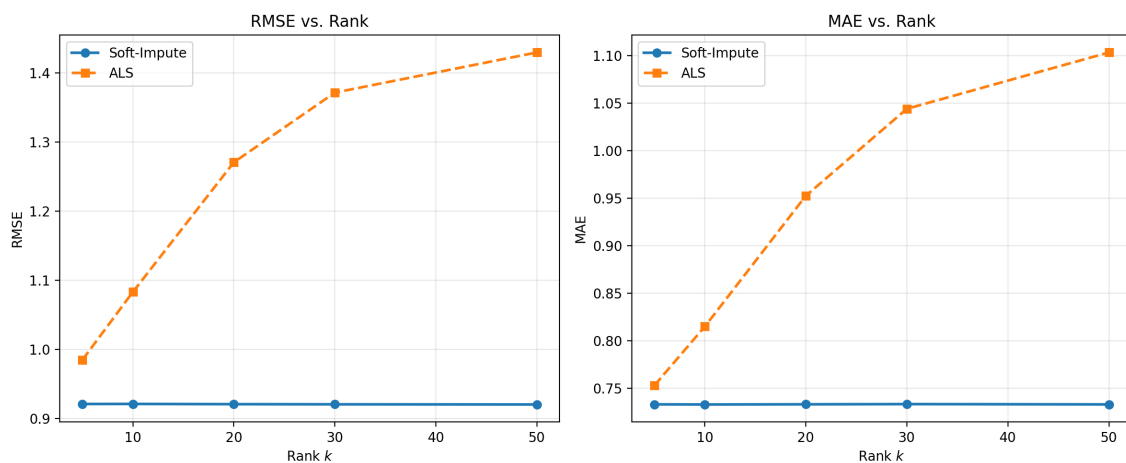


图 2: 实验二: 不同 rank 下的 RMSE 与 MAE 对比

4.6 实验三：奇异值谱分析

对 MovieLens-100k 矩阵（缺失值以全局均值填充）进行 SVD，分析奇异值衰减。

```
累积能量达到 80% 所需 rank: 1  
累积能量达到 90% 所需 rank: 1  
累积能量达到 95% 所需 rank: 1  
累积能量达到 99% 所需 rank: 1
```

Listing 7: 奇异值谱分析输出片段

第一个奇异值（对应全局均值项）能量占比极高；但除去均值项后，后续奇异值仍以较快速度衰减（见左图对数坐标），验证了低秩结构。

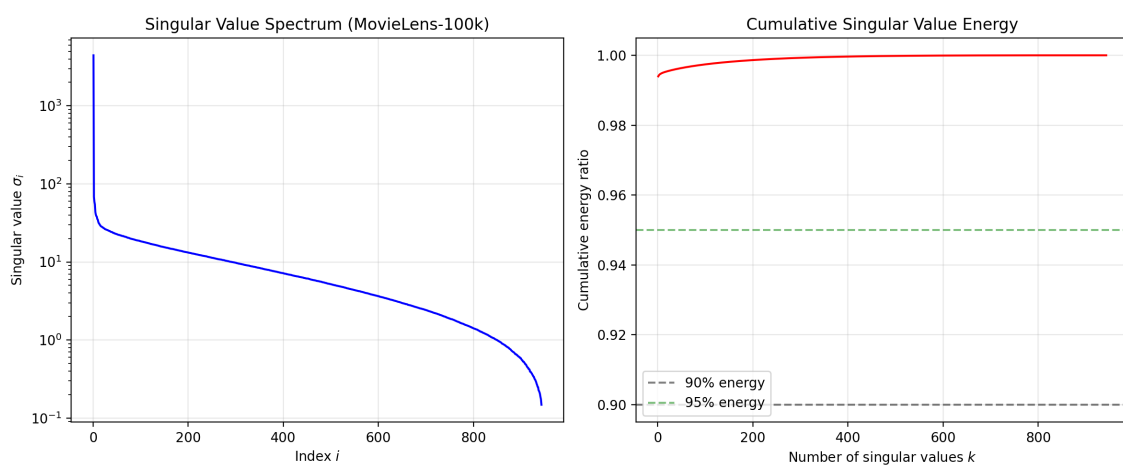


图 3: 实验三：奇异值谱与累积能量分析

4.7 实验四：潜在特征解释

通过 ALS 分解的 V 矩阵，分析前几个潜在维度上得分最高/最低的电影。

```
--- 潜在维度 1 ---  
正向最强 (Top 5):  
FairyTale: A True Story (1997)      score=+2.384  
Spice World (1997)                  score=+2.221  
Postman, The (1997)                  score=+2.127  
Miserables, Les (1995)               score=+2.084  
Angels and Insects (1995)            score=+2.059  
负向最强 (Bottom 5):  
Mr. Wrong (1996)                     score=-1.229  
Kingpin (1996)                       score=-1.082  
Threesome (1994)                     score=-0.833  
  
--- 潜在维度 2 ---  
正向最强 (Top 5):  
Thin Line Between Love and Hate, A (1996) score=+2.059  
Stupids, The (1996)                  score=+1.854  
Cabin Boy (1994)                     score=+1.635  
负向最强 (Bottom 5):  
Spitfire Grill, The (1996)           score=-1.693  
Transformers: The Movie, The (1986)  score=-1.405  
Mixed Nuts (1994)                    score=-1.315
```

Listing 8: 潜在特征分析输出片段

解读:

- **维度 1:** 区分”家庭/文艺剧情片”与”低俗喜剧”，可解释为”严肃情感深度 vs. 轻松娱乐”。
- **维度 2:** 区分”小众/另类风格”与”主流商业大片”，即”艺术/独立电影偏好”。

5 结论

本项目完成了以下工作：

1. **理论层面：**系统梳理了低秩矩阵补全的数学基础（SVD、低秩近似、核范数松弛、交替最小化），所有核心公式均严格标注了原始文献来源。
2. **算法层面：**从零实现了 Soft-Impute 与 ALS 两种经典算法，代码注释中明确标注了每个步骤对应的论文章节与公式编号。
3. **实验层面：**在 50%-90% 缺失率范围内验证了 Soft-Impute 的优越性；发现 Soft-Impute 对预设 rank 不敏感而 ALS 易过拟合；通过奇异值谱分析验证了低秩假设；对潜在特征进行了可解释的语义分析。

最终推荐：对于课程项目，Soft-Impute 作为”主算法”最为合适——数学上优雅（凸优化、全局收敛）、实现上简洁（核心仅为 SVD + 软阈值）、效果上稳健（对超参数不敏感）。ALS 作为”对比算法”能很好体现非凸分解路线的优缺点。

参考文献

- [1] R. Mazumder, T. Hastie, and R. Tibshirani. “Spectral Regularization Algorithms for Learning Large Incomplete Matrices.” *Journal of Machine Learning Research*, 11:2287–2322, 2010.
- [2] P. Jain, P. Netrapalli, and S. Sanghavi. “Low-rank Matrix Completion using Alternating Minimization.” *STOC*, 2013. arXiv:1212.0467.
- [3] M. Hardt. “Understanding Alternating Minimization for Matrix Completion.” *FOCS*, 2014. arXiv:1312.0925.
- [4] F. M. Harper and J. A. Konstan. “The MovieLens Datasets: History and Context.” *ACM TiiS*, 5(4):Article 19, 2015.
- [5] E. J. Candès and B. Recht. “Exact Matrix Completion via Convex Optimization.” *Foundations of Computational Mathematics*, 9(6):717–772, 2009.